

Шпаргалка по Terraform

Инфраструктура как код: команды, язык описания, переменные и выходные значения, состояние и его хранение, модули, зависимости и приёмы безопасной работы с рабочими средами.

Основные понятия

- **Провайдер** — плагин для работы с платформой (AWS, Google Cloud, Kubernetes).
- **Ресурс** — управляемый объект инфраструктуры.
- **Источник данных** (data) — чтение того, что создано вне этой конфигурации.
- **Состояние** — файл соответствия между описанием и реальными объектами.
- **План** — предполагаемые изменения до их применения.
- **Модуль** — набор ресурсов, оформленный для повторного использования.

Команды

```
terraform init
  Скачать провайдеры и модули, настроить хранилище
  состояния

terraform fmt -recursive / terraform validate
  Форматирование и проверка синтаксиса

terraform plan -out=tfplan
  Показать изменения и сохранить план

terraform apply tfplan
  Применить именно то, что показал план

terraform apply -target=aws_instance.web
  Только один ресурс — крайняя мера при разборе
  проблем

terraform destroy
  Удалить всё описанное конфигурацией

terraform output / output -json
  Выходные значения, в том числе для других
  инструментов

terraform show / state list
  Текущее состояние; перечень ресурсов в нём

terraform import адрес id
  Взять под управление существующий объект

terraform state mv / rm
  Переименовать ресурс в состоянии; забыть о нём, не
  удаляя в облаке
```

Конфигурация

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  backend "s3" {
    bucket = "tf-state"
    key    = "prod/app.tfstate"
    region = "eu-central-1"
  }
}

provider "aws" {
  region = var.region
}

resource "aws_s3_bucket" "assets" {
  bucket = "${var.project}-assets"
  tags   = local.common_tags
}
```

Переменные и значения

```
variable "region" {
  type      = string
  default   = "eu-central-1"
}

variable "instance_count" {
  type      = number
  validation {
    condition     = var.instance_count > 0
    error_message = "Должно быть больше нуля."
  }
}

locals {
  common_tags = {
    project = var.project
    managed = "terraform"
  }
}

output "bucket_name" {
  value = aws_s3_bucket.assets.bucket
}
```

```
terraform.tfvars / -var-file=prod.tfvars
  Значения переменных для среды
```

```
TF_VAR_region=eu-west-1
  Переменная через окружение — удобно в CI
```

```
sensitive = true
  Не выводить значение в журнал
```

Выражения

```
count = 3 / count.index
    Несколько одинаковых ресурсов

for_each = toset(["a", "b"])
    По набору или карте; each.key, each.value. Устойчивее
    count при изменениях списка

dynamic "ingress" { for_each = ... }
    Повторяющиеся вложенные блоки

condition ? a : b
    Условный выбор значения

depends_on = [aws_iam_role.app]
    Явная зависимость, когда неявной недостаточно

lifecycle { prevent_destroy = true }
    Защита от удаления; есть также create_before_destroy
    и ignore_changes

templatefile("init.sh.tftpl", { port = var.port })
    Шаблон с подстановкой значений
```

Модули

```
module "network" {
    source = "../modules/network"

    cidr      = "10.0.0.0/16"
    project = var.project
}

module "db" {
    source = "terraform-aws-modules/rds/aws"
    version = "~> 6.0"

    identifier = "app-db"
    subnet_ids = module.network.private_subnets
}
```

Раскладка модуля: main.tf, variables.tf, outputs.tf, README.md. Версию внешнего модуля фиксируйте — иначе сборка перестанет быть воспроизводимой.

Состояние

- Храните состояние удалённо (S3 с блокировкой, GCS, Terraform Cloud) — локальный файл не подходит для команды.
- Блокировка не даёт двум людям применять изменения одновременно.
- Состояние содержит значения ресурсов, в том числе чувствительные: доступ к нему ограничивайте, включайте шифрование.
- Отдельное состояние на среду (dev, stage, prod) — либо рабочие пространства, либо разные ключи хранения.
- Правьте состояние только командами terraform state, не текстовым редактором.

```
terraform workspace new prod / select prod
    Рабочие пространства: одна конфигурация, разные
    состояния
```

```
terraform force-unlock ID
    Снять зависшую блокировку — только когда точно
    известно, что процесс мёртв
```

Отладка

```
terraform plan -refresh=false
    Быстрый план без опроса облака

terraform console
    Проверить выражение и посмотреть значения
    интерактивно

TF_LOG=DEBUG terraform apply
    Подробный журнал (уровни TRACE, DEBUG, INFO)

terraform graph | dot -Tsvg > graph.svg
    Схема зависимостей ресурсов

terraform providers lock
    Зафиксировать контрольные суммы провайдеров для
    всех платформ
```

Работа в команде

- План — обязательная часть код-ревью: смотрят не только на код, но и на то, что он изменит.
- В конвейере CI: fmt и validate на каждый запрос, plan с публикацией результата, apply — по подтверждению.
- Никаких изменений руками в консоли облака: расхождение всплывёт в ближайшем плане как неожиданное удаление.
- Секреты — во внешнем хранилище, а не в tfvars в репозитории.
- Версии Terraform и провайдеров фиксируйте: обновление версии — отдельное изменение.
- Проверки безопасности конфигурации (tfsec, Checkov) полезны на этапе запроса на слияние.

Опасные места

- План показывает -/+ replace — ресурс будет пересоздан; для базы данных это потеря данных.
- Смена name или другого неизменяемого поля почти всегда означает пересоздание.
- Переход с count на for_each меняет адреса ресурсов: помогает terraform state mv или блок moved.
- terraform destroy в рабочей среде — самая дорогая опечатка; защищайте ресурсы prevent_destroy.
- Удаление ресурса из кода означает его удаление в облаке при следующем применении.